

Table of Contents

Studium — Infrastructure Specification

Subsystem 7 of 7. Version 1.0. Status: build-ready.

The last spec in the seven-subsystem sequence. Written against data layer v1.1 (with v1.2 pending, nineteen items batched across prior subsystems), agent runtime v1.0 with v1.0.1 patch and v1.0.2 amendment applied (with v1.1 pending, eight items), retrieval v1.0 (with v1.1 pending, four to five items), frontend v1.0, ingestion v1.0, and evaluation v1.0. None of the pending items affect the infrastructure boundary. This subsystem specifies what runs where, how it's monitored, how it recovers from failure, and how it grows beyond MVP scale.

1. Overview

This subsystem specifies the operational envelope in which the six preceding subsystems run. Where every prior spec answered “what does this component do,” this one answers “how does the running system stay running, get observed, recover from failure, and grow.” A senior engineer with all six prior specs and this document should be able to deploy Studium from a clean environment, connect the observability stack, verify backups, and hand the reviewer credentials and dashboards that let them operate the system day-to-day.

Infrastructure is deliberately last in the spec sequence because it's the subsystem most likely to be re-decided under real production experience. Deployment topology, observability tooling, alert thresholds — all of these are educated first guesses that the first months of production use will sharpen. What this spec locks is the shape and the boundary responsibilities; what a v1.1 revision would tune is the specific numbers.

The spec is smaller than any prior — around 22 pages — because infrastructure is mostly configuration and integration rather than new behavior. Every choice here is defensible against alternatives, but few of the choices are load-bearing in the way that agent-runtime state machine choices or ingestion invariants are.

2. Scope and non-goals

In scope.

- Deployment topology: what runs where, at what size, in which region.
- Secrets management: what's a secret, where it lives, how it rotates.
- Observability: Langfuse for LLM tracing, Sentry for errors, OpenTelemetry for HTTP, Fly.io metrics for infrastructure.
- Alerts and paging: what conditions fire alerts, who receives them, escalation.

- Backups and disaster recovery: what gets backed up, on what schedule, how restore is tested.
- Signing key management: Ed25519 keys for portfolio items, rotation, publication.
- CI/CD: what exists today, what to add, deployment promotion.
- Cost monitoring: infrastructure cost tracking (distinct from the cost_ledger which tracks LLM cost).
- Scaling triggers: when to move beyond single-node topology.
- Retention operations: the cron infrastructure that executes data layer §10's retention policies.
- Failure modes and recovery procedures.

Explicitly out of scope.

- Business continuity beyond the technical stack (legal entity, insurance, vendor contracts). Infrastructure describes the running system; corporate structure is separate.
- Multi-region deployment. Toronto (yyz) only for MVP. Multi-region is a v2 concern once real learner geography justifies it.
- Enterprise SSO, SAML, on-premise deployment. Studium is a single-tenant hosted product at MVP.
- Compliance certifications (SOC 2, ISO 27001). Meaningful only at scales that require them; not MVP concerns.
- Load testing methodology and capacity planning at production scale. §14 specifies scaling triggers; the load testing that would inform them is a separate engineering exercise beyond the spec.
- Detailed cost projections at classroom or university tier. §13 gives MVP numbers; scaling costs are estimated but not authoritative.

3. Design principles

One region, one primary, no premature distribution. MVP runs in one Fly.io region (Toronto, yyz) with a single backend instance, a single frontend instance, and a single Postgres primary. Every attempt to distribute for reliability creates new failure modes to handle, new consistency questions to answer, and new cost lines to justify. Distribution happens when the reliability benefit is measurably worth the complexity — not before.

Managed services where they exist; self-hosted only where they don't. Fly Postgres, Fly volumes, Cloudflare R2 (when it's needed), Langfuse cloud, Sentry SaaS. The cost of running these services well is not what Studium exists to solve; paying the managed-service premium buys back engineering time. Exceptions are called out explicitly (§4.3 explains why we don't use Vercel for the frontend).

Secrets are secrets, not configuration. API keys, database URLs, signing keys, session secrets — anything whose accidental disclosure would compromise the system — lives in Fly secrets, never in code, never in committed environment files, never in logs. The

distinction between “configuration that can be committed” (feature flags, timeouts, model choices) and “secrets that cannot” is bright and enforced.

Observability is designed in, not retrofitted. Every LLM call goes to Langfuse. Every HTTP request produces an OpenTelemetry span. Every unhandled exception goes to Sentry. Metrics are structured; log messages include correlation IDs; the reviewer can navigate from a specific learner turn to the LLM trace that produced it, to the HTTP request that carried it, to the error (if any) that fired. The story is built up-front, not added when a debugging emergency reveals its absence.

Backups exist to be restored, not to sit unused. Any backup mechanism has a restore procedure and the restore procedure is exercised regularly. Untested backups are a false comfort; a backup that has never been restored is a coin flip that could have been avoided by testing.

Alerts fire on what’s actionable, not what’s changing. Not every metric change warrants attention. Alerts exist for conditions where the reviewer needs to do something — queue depth exceeding capacity, cost approaching a cap, error rate spiking, uptime failing. Metric dashboards exist for exploratory investigation. Confusing the two produces alert fatigue and hides the alerts that matter.

Cost is measured, not estimated. Both LLM cost (via `cost_ledger`, subsystem 2 §19 and data layer §6.12) and infrastructure cost (via Fly billing plus provider invoices) get tracked with actual numbers. Projections in this spec are for planning; the numbers that matter are the ones the running system produces.

Rotation is scheduled, not reactive. Signing keys rotate on a calendar (annually) whether or not there’s a reason. API keys rotate on a calendar (quarterly) or immediately on suspicion of exposure. Establishing rotation as routine means the operational muscle exists when a rotation becomes urgent; skipping rotation “because nothing’s wrong” is how the muscle atrophies.

4. Deployment topology

4.1 Region

Fly.io Toronto (yyz). Chosen because the initial user base is in Ontario (you at OTU, expected initial learners at Ontario universities). Latency from Toronto to end users in eastern Canada is 15-40ms; latency to Anthropic’s us-east endpoints from Toronto is 40-80ms; neither dominates the per-turn budget. If Studium later serves European or Asian users at meaningful volume, adding a region is a v2 concern.

4.2 The backend

Fly.io Machines VM. One instance for MVP. Configuration:

- **Size:** shared-cpu-2x with 2GB RAM. Enough headroom for the FastAPI process, embedding worker, ingestion worker, and the scheduled retention/warming jobs

without CPU or memory pressure at MVP scale (up to ~20 concurrent sessions, ~10 concurrent ingestion jobs).

- **Auto-scaling:** Disabled for MVP. Auto-scaling introduces cold-start latency that hurts the streaming interruption UX; single instance is faster and simpler at MVP scale.
- **Restart policy:** Automatic on unhandled exception (Fly's default). No manual intervention needed for ordinary crashes; Sentry captures the exception, the instance restarts within seconds.
- **Health check:** GET /health returns 200 when the FastAPI process is healthy (checked separately for Postgres connection and Anthropic reachability); Fly's load balancer routes only to healthy instances.

4.3 The frontend

Fly.io Machines VM, separate app. One instance for MVP. Configuration:

- **Size:** shared-cpu-1x with 1GB RAM. Next.js production build is lightweight; the frontend does no LLM calls itself, no heavy computation, only serves static assets and streams from the backend.
- **Auto-scaling:** Disabled, same reasoning as backend.

Why Fly and not Vercel for the frontend. Vercel would be the natural choice for a Next.js app, and at scale it's likely correct. For MVP, keeping frontend and backend on the same platform (Fly) simplifies deployment, secrets management, and network configuration. Vercel's SSR streaming has occasional edge cases with SSE-from-backend patterns that would need debugging; keeping both apps in one Fly project avoids the debugging.

4.4 Postgres

Fly Postgres, single primary. Configuration:

- **Version:** Postgres 16 with pgvector 0.8+ extension.
- **Size:** shared-cpu-2x with 4GB RAM and 40GB attached volume for MVP. The 40GB accommodates: schema data (small), source_chunks and embeddings (grows to ~1GB per 100 sources), agent_traces and session_turns (grows with usage), cost_ledger and mastery_events. Room for a year of MVP-scale growth.
- **Backups:** Fly's automated daily backup + point-in-time recovery (retained 14 days for the default plan).
- **Replicas:** None for MVP. Read replicas become worth it when the backend's read query latency exceeds targets (rare at MVP query volume) or when the backup restore is too slow (14-day PITR is fast enough).

4.5 Object storage

Cloudflare R2, provisioned but not used until needed. For MVP, source PDFs and extracted intermediate artifacts live on the Fly volume attached to the backend. When total corpus storage approaches 30GB (roughly 300 sources at ~100MB each including

artifacts), migration to R2 happens as a background job. R2 is chosen over S3 for egress cost (R2 has no egress fees; S3 charges per GB out).

Storage abstraction. All code accesses source files via `studium.storage` interface. Local implementation reads from the Fly volume; R2 implementation reads via S3-compatible API. Migration is a configuration change plus a one-time data migration, not a code change.

4.6 Redis or Memcached

Not deployed for MVP. The retrieval subsystem §14 noted in-process caching is sufficient at single-node scale. When Studium goes multi-node (auto-scaling turned on, or explicit multi-instance for redundancy), the retrieval cache and the intent classifier's small caches move to Redis. Fly's managed Redis is the natural choice; Upstash is the alternative.

4.7 The network

Fly's private network. Backend and Postgres communicate over Fly's internal `.internal` DNS with no public exposure. Frontend proxies API calls to backend via internal DNS. Only the frontend has a public IP; the backend is reachable only via the frontend's proxy or Fly's `flyctl proxy` for admin access.

Public endpoints:

- `https://studium.app` — frontend (learner-facing)
- `https://studium.app/api/*` — proxied to backend
- `https://studium.app/verify/{portfolio_item_id}` — public portfolio verification (per evaluation §12.4)

No other public endpoints. Admin surfaces (reviewer CLI, database shell) are accessed via `flyctl` from authorized workstations.

5. Environments

5.1 Local development

Docker Compose for Postgres with `pgvector`; native processes for backend and frontend. The dev's `backend/docker-compose.yml` is the canonical local setup. Local Anthropic/Voyage calls use the same production API keys against a `STUDIUM_ENV=local` flag that isolates local activity in Langfuse traces.

5.2 Staging

Not deployed for MVP. The rationale: a single-user MVP with an active reviewer (Yannis) doesn't gain enough from staging to justify the operational overhead. When multiple learners are onboarded and changes need pre-production verification, staging becomes worth adding. It would mirror production topology at smaller size (one shared-cpu-1x for each service).

5.3 Production

The Fly deployment described in §4. Single environment for MVP, with the main branch as the deployable state.

5.4 Preview deployments

Fly's preview environment support (via `flyctl deploy --strategy blue-green`) enables preview URLs for a PR branch to be validated against real infrastructure without touching production. Not enabled by default for MVP; can be turned on when a specific PR warrants it.

6. Secrets management

6.1 What is a secret

Anything whose accidental disclosure would either (a) allow unauthorized access to systems, (b) enable financial fraud, or (c) compromise learner data. Concretely:

- API keys: Anthropic, Voyage, Sentry, Langfuse
- Database URLs (contain credentials)
- Session signing secret (used for cookie authentication)
- Portfolio Ed25519 signing keys (both current and any recent expired keys retained for grace period)
- Cloudflare R2 credentials (when provisioned)
- Any OAuth client secrets (none at MVP; if added later)

6.2 Where secrets live

Production: Fly secrets (`flyctl secrets set`). Encrypted at rest, injected into the runtime environment. Never visible in logs, never returned in API responses, never in error messages.

Local development: `.env.local` file, gitignored. A `.env.example` file (committed) lists variable names with placeholder values so contributors know what to populate.

CI: GitHub Actions secrets. Same list as production (needed for Tier 3 tests that call real APIs). Scoped to specific workflows; not exposed to workflows triggered by PRs from forks (a security consideration for when the repo might have outside contributors — deferred, not MVP).

6.3 Rotation

Anthropic and Voyage API keys. Quarterly rotation on a calendar reminder, or immediately on suspicion of exposure. Procedure:

1. Generate new key in provider console.
2. `flyctl secrets set ANTHROPIC_API_KEY=<new>` (deploys with the new key).
3. Verify a session start succeeds against the new key.

4. Revoke the old key in provider console.
5. Update GitHub Actions secret with the new key.
6. Update local `.env.local` on your workstation.

Total time: ~10 minutes per rotation, done every three months.

Signing keys. Annually. Procedure detailed in §11.

Session signing secret. Rotated when auth infrastructure gets rebuilt (currently a placeholder per subsystem 4 §14; real auth is a v1.1 concern for the frontend spec).

Database URL. Never rotates unless Postgres credentials are reset (which is a Fly-side operation, rarely needed).

6.4 Access

The list of humans who have secret access is small and audited:

- **Yannis:** all secrets (product owner, sole operator at MVP).
- **The dev:** all secrets during active build cycles; access reviewed at end of each build (currently the arrangement).
- **Nobody else** at MVP.

When a person's access is revoked, all secrets they had access to are rotated within 48 hours. This is the "assume compromise" default even when there's no evidence of it.

7. Observability

Four systems, each with a specific role. They share correlation IDs so navigation across them is possible.

7.1 Langfuse (LLM tracing)

What it captures. Every LLM call — the system prompt hash, the messages, the completion, tokens in/out/cached, latency, cost, model, agent identity, session_id, user_id. Every trace is nested per subsystem 2 §22's hierarchy: session → turn → generation.

Configuration. Langfuse Cloud (SaaS), self-hosted deployment is a v2 option. Project per environment (local, production). Traces retained for 90 days on the free tier; extended retention purchased if analytical needs justify it.

Dashboards. Configured in Langfuse per subsystem 2 §22's dashboard list — per-agent latency, cache hit rate, cost, review-queue flag rate. Reviewer accesses via Langfuse's web UI.

7.2 Sentry (errors and performance)

What it captures. Every unhandled exception in backend and frontend. Performance traces for a sampled subset of requests (10% at MVP; sampling rate adjustable).

Configuration. Sentry SaaS, one organization, one project per app (backend, frontend). PII scrubbing enabled — learner utterances, session content, and cost figures are stripped from error contexts before send.

Alerts. Configured in Sentry (per §8): error rate exceeding threshold, new error types appearing, specific critical exceptions (BudgetExceededError firing outside expected pattern, MissingProvenance from the ingestion invariant).

7.3 OpenTelemetry (HTTP request tracing)

What it captures. Every FastAPI request produces a span. Every Next.js server-side render produces a span. Trace IDs propagate between frontend and backend so a slow session can be traced end-to-end.

Configuration. OpenTelemetry SDK in both apps. Traces export to Langfuse (which accepts OTel format) so LLM traces and HTTP traces share the same trace ID. Metric export to Fly.io's built-in metrics.

No separate tracing backend (Jaeger, Honeycomb). Adding one is a v2 concern if Langfuse's HTTP trace view proves insufficient.

7.4 Fly.io metrics (infrastructure)

What it captures. CPU, memory, disk, network per instance. Postgres query stats, connection count, storage growth. Automatic.

Dashboards. Fly's built-in dashboards plus a custom Grafana instance if metric visualization needs to be shared with someone who doesn't have Fly access. Grafana deferred until it's needed.

Uptime monitoring. External service (Uptime Robot or Better Stack, free tier) pings <https://studium.app/health> every 5 minutes. Failure fires an alert per §8. This is the only third-party monitoring outside the Fly-Sentry-Langfuse stack; it exists because trusting Fly to monitor itself creates a single point of failure.

8. Alerts and paging

8.1 What fires alerts

Alerts are for conditions that require the reviewer to do something. Not every metric change warrants attention.

Condition	Threshold	Delivery	Response time expected
Uptime failure	2 consecutive 5-min pings fail	Email + push	Immediate
Sentry error rate spike	10× baseline over 15 min	Email	Within 1 hour
Any BudgetExceededError	Any occurrence	Email	Within 4 hours

Condition	Threshold	Delivery	Response time expected
firing			
MissingProvenance from ingestion	Any occurrence	Email	Within 24 hours
Content review queue depth	> 20 pending items	Email (daily digest)	Within 48 hours
Ingestion review queue depth	> 20 pending items	Email (daily digest)	Within 48 hours
Postgres storage	> 80% of 40GB volume	Email	Within 1 week
Fly VM memory	> 90% sustained 15 min	Email	Within 4 hours
Anthropic API 5xx rate	> 5% over 15 min	Email	Immediate (nothing to do but wait for provider)
Voyage API 5xx rate	> 5% over 15 min	Email	Immediate
Weekly evaluation regression failure	Any dataset regressing beyond tolerance	Email	Within 1 week
Daily cost trending toward monthly cap	Extrapolated month-end > 90% of cap	Email (daily)	Within 1 week

8.2 What does not fire alerts

- Individual failed retrievals (thin grounding is a queue item, not an alert)
- Individual failed agent calls (retries handle these; only sustained failure patterns alert)
- Learner-visible degradation events (the learner sees them; the reviewer sees the queue item; alerting on every one would produce noise)
- Session terminations (idle timeouts, budget caps, learner-initiated closes are normal)
- Preview deploy failures (developer sees the failure in their own workflow)

8.3 Delivery

MVP: Email to Yannis. No paging service, no SMS, no on-call rotation. The response times in §8.1 reflect this — nothing is truly “immediate” because the operator sleeps.

When to add paging: When either (a) real learners other than Yannis start using the system for their real coursework, so uptime failures have real academic consequences, or (b) revenue depends on uptime and downtime has measurable cost. Neither applies at MVP.

Escalation: Not defined for MVP because there's no one to escalate to. When a second operator joins, escalation becomes a real question worth answering.

9. Backups and disaster recovery

9.1 What gets backed up

Postgres. Fly Postgres automated daily backups + point-in-time recovery (14-day retention on default plan). Backups are encrypted at rest.

Fly volume (source files). Fly automatic snapshots (daily, 7-day retention). Manual snapshot before any risky operation.

Signing keys. Backed up separately as encrypted files stored in Yannis's password manager. This is the one thing whose loss cannot be recovered from the running system — a lost signing key means every portfolio item signed with it becomes unverifiable. Two independent copies.

Code and content. Git repository at GitHub. GitHub's own redundancy is sufficient; additional backup (self-hosted mirror) is a v2 concern.

What is not backed up. Log files (Sentry retains 90 days; Langfuse retains 90 days). Uptime monitoring history (Uptime Robot retains 30 days). These are considered operational data, not durable data.

9.2 Restore procedures

Postgres restore. Documented procedure in `docs/ops/postgres-restore.md` (to be written during infrastructure build). Steps: choose restore point via Fly console, initiate restore into a new Postgres instance, verify data integrity via `studium ops verify-restore`, swap `DATABASE_URL` to point at the restored instance, redirect traffic. Total time: 15-45 minutes depending on data volume.

Volume restore. Similar procedure via `flyctl volumes restore`. Faster (10-15 minutes typical).

Signing key restore. Retrieve encrypted backup from password manager, decrypt, deploy via `flyctl secrets set`. Verify with a portfolio item signing round-trip. 5-10 minutes.

9.3 Restore drill schedule

Quarterly. Once per quarter, execute a full restore drill:

1. Restore Postgres to a scratch instance (does not touch production).
2. Verify data integrity via `studium ops verify-restore`.
3. Attempt to launch a backend instance against the restored database.
4. Confirm the backend starts and can serve a health check.
5. Tear down the scratch instance.

6. Document the drill in `docs/ops/drill-log.md` with date, restore time, any issues encountered.

The point is not the specific procedure — the point is that backup mechanisms decay silently and only exercise proves they still work. A backup that has never been restored is unverified.

9.4 Disaster scenarios and RPO/RTO

RPO (Recovery Point Objective): how much data loss is acceptable. Target: 24 hours (daily backup granularity). Point-in-time recovery reduces this to 5 minutes for Postgres.

RTO (Recovery Time Objective): how long recovery takes. Target: 4 hours for full production restore. Realistic based on the procedures above.

Neither number matters at MVP scale (one user, no revenue tied to uptime). Both matter progressively more as learners depend on the system for real coursework.

10. CI/CD

10.1 What exists

Per prior subsystem specs and the dev's implementation:

- `.github/workflows/data-layer.yml` — Tier 1 always, Tier 2 on push to main and yannis (or on PR touching schema/migration paths)
- `.github/workflows/agent-runtime.yml` — same pattern for runtime paths
- `.github/workflows/retrieval.yml` — same pattern for retrieval paths
- `.github/workflows/frontend.yml` — same pattern for frontend paths

Each workflow: checkout, Python/Node install, dependencies, offline tier, decide-if-online, online tier if applicable, cleanup.

Tier 3 (paid) runs on-demand with `STUDIUM_RUN_PAID_TESTS=1` environment variable and requires the API secrets set at workflow scope.

10.2 What to add

Per-subsystem workflows for the remaining subsystems:

- `.github/workflows/ingestion.yml` — when subsystem 5 build lands
- `.github/workflows/evaluation.yml` — when subsystem 6 build lands
- `.github/workflows/infrastructure.yml` — deployment pipeline (see §10.3)

Each follows the established shape.

Regression gate workflow for prompt changes:

- `.github/workflows/prompt-regression.yml` — runs the evaluation harness (subsystem 6 §13) against affected datasets when prompts change. Blocks merge if regression exceeds tolerance and reviewer hasn't approved.

10.3 Deployment pipeline

Manual promotion for MVP. Merges to main do not auto-deploy. The operator runs `flyctl deploy` manually after verifying CI green. This is safer than auto-deploy for a project where the operator is human, single, and prefers a moment of intention before shipping.

Migrations run on deploy. Alembic migrations execute as part of `flyctl deploy` via a release command. Migration failure aborts the deploy; the previous version continues serving.

Post-deploy verification. After deploy completes, run `studium ops smoke-test` which exercises: health check, database connectivity, Anthropic reachability, Voyage reachability (when provisioned), one full session round-trip against a test user. Failure alerts.

10.4 Rollback

Instant rollback via Fly. `flyctl releases rollback` reverts to the previous release. Takes ~30 seconds.

Database rollback. Alembic downgrade is available but rarely the right choice — data changes typically can't be safely reversed. Rollback strategy is “roll forward with a fix” for schema changes; for pure code changes, `flyctl releases rollback` is sufficient.

11. Signing key management

Per evaluation subsystem §12, portfolio items are signed with Ed25519 keys. This section covers how those keys live in the running system.

11.1 Key generation

Initial key generated via `studium ops mint-signing-key`, which produces a keypair and stores:

- Private key: `SIGNING_KEY_CURRENT` Fly secret + encrypted backup in Yannis's password manager
- Public key: `signing_keys` table row with `active = TRUE` (per data layer §6.10)

Only one active signing key at any time. Historical public keys remain in the table (`active = FALSE`) for verifying old portfolio items.

11.2 Rotation

Annually. Procedure:

1. Generate new keypair.

2. Insert new row in `signing_keys` with `active = FALSE` initially.
3. Update `SIGNING_KEY_CURRENT` Fly secret to the new private key.
4. Flip the new row to `active = TRUE`; flip the old row to `active = FALSE` in the same transaction.
5. Verify next portfolio item issuance uses the new key.
6. Old private key: retained in encrypted backup for 90 days (in case of dispute), then destroyed.
7. Old public key: retained indefinitely in `signing_keys` for verification of historical portfolio items.

11.3 Publication

Public keys are published at <https://studium.app/api/signing-keys> as a JSON array. Verifiers fetch this to obtain the key needed to verify a portfolio item. The endpoint is unauthenticated and cacheable.

11.4 Compromise response

If a signing key is suspected compromised:

1. Immediately rotate per §11.2 (bumped to hours, not scheduled).
2. Mark the compromised key with a `compromised_at` timestamp (adds a column to `signing_keys`; joins v1.2 batch as the twentieth item).
3. Publish notice to any downstream verifiers explaining that portfolio items signed with the compromised key between the earliest possible compromise and rotation should be treated with additional scrutiny.
4. Investigate scope: what accessed the key, when, from where. Sentry logs plus Fly access logs are the primary sources.

Compromise has never happened in the project's history. This procedure exists so it's answered before it does.

12. Retention operations

Data layer §10 defines retention policies. This section specifies the operational infrastructure that executes them.

12.1 The retention worker

Runs as a scheduled task (cron-like) in the backend process. Fires daily at 02:00 UTC. Iterates tables with retention policies, deletes rows past retention, logs actions.

12.2 Retention log

Every retention action writes to `retention_actions` (a new table joining v1.2 batch as the twenty-first item):

```
CREATE TABLE retention_actions (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v7(),
```

```

    ran_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    table_name      TEXT NOT NULL,
    rows_deleted    INTEGER NOT NULL,
    duration_ms     INTEGER NOT NULL,
    metadata        JSONB NOT NULL DEFAULT '{}'::jsonb
);

CREATE INDEX idx_retention_actions_recent
ON retention_actions (ran_at DESC);

```

Audit trail for any question of “why did that data go away” — the answer is either “retention policy X, on date Y, deleted N rows” or “something else happened, investigate.”

12.3 Erasure operations

Distinct from retention. When a learner requests erasure (data layer §10 step 2), the operation runs immediately (not on the daily schedule) via `studium ops erase-user <user_id>`. The dev’s earlier fix to the erasure procedure (the V13 resolution around the cost ledger merge) is what this command executes.

12.4 Manual retention overrides

Occasionally a specific dataset needs to be retained beyond policy for legitimate reasons (evidence in a rights dispute, research study, longitudinal analysis). Applied via `studium ops set-retention-hold <table> <row_id> <reason>` which prevents the retention worker from deleting that specific row. Documented in `retention_holds` (a small table, joining v1.2 batch as the twenty-second item).

13. Cost monitoring

Two distinct cost surfaces: LLM costs (already tracked in `cost_ledger` per data layer §6.12) and infrastructure costs (Fly, Cloudflare, third-party services).

13.1 LLM cost visibility

Reviewer accesses via `studium ops cost-report --start <date> --end <date>` which produces:

- Total cost by user
- Total cost by cost line (agent, ingestion, evaluation, content, storage)
- Trend over time
- Projection to end of billing period
- Anomaly detection: any user whose spend is $> 3\times$ their trailing 7-day average

13.2 Infrastructure cost

Not integrated with `cost_ledger` (which tracks per-user LLM cost). Tracked separately via monthly review of provider invoices:

- Fly.io: ~\$50-100/month for MVP topology (backend + frontend + Postgres + volumes)
- Cloudflare R2: \$0 for MVP (not provisioned yet); ~\$5-15/month when needed
- Langfuse: \$0 for MVP (free tier); ~\$50/month if extended retention needed
- Sentry: \$0 for MVP (free tier); ~\$25/month at classroom scale
- Voyage: usage-based, tracked in cost_ledger
- Anthropic: usage-based, tracked in cost_ledger

Total infrastructure cost projection:

- MVP: ~\$50-100/month + LLM usage
- Classroom (10 subjects, 30 users): ~\$200-400/month + LLM usage
- University (100 subjects, 500 users): ~\$1000-2000/month + LLM usage

Actual numbers become known once the system runs.

13.3 Cost alerts

Per §8.1: daily extrapolation of month-end LLM spend against cap fires an alert when projected end-of-month exceeds 90% of cap.

Infrastructure cost has no automated alerts at MVP (invoices are small, monthly review is sufficient). When infrastructure cost exceeds \$500/month, automated alerting becomes worth adding.

14. Scaling triggers

MVP topology is deliberately conservative. Triggers for scaling up:

14.1 Backend instances

Trigger: sustained CPU > 70% over a 15-min window during peak hours, OR concurrent sessions consistently > 20.

Action: enable auto-scaling with min=1, max=3. Watch the interruption UX for cold-start latency issues.

14.2 Postgres

Trigger: query latency at p95 exceeding subsystem 1 §14 targets (currently sub-10ms warm cache), OR storage > 80% of volume.

Actions: first, add read replicas. Second, if writes are the bottleneck, tune Postgres config. Third, upgrade Postgres instance size.

14.3 Object storage

Trigger: source volume > 30GB.

Action: migrate to R2. Migration procedure documented in docs/ops/storage-migration.md.

14.4 Multi-region

Trigger: substantial user population outside eastern North America, OR uptime requirements too strict for single-region.

Action: add secondary region, replicate Postgres, configure Fly's global load balancing. Real engineering work; a v2 concern.

14.5 Redis

Trigger: multi-instance backend deployment (which necessitates shared cache for retrieval and intent classification).

Action: add Fly managed Redis, migrate in-process caches to Redis-backed. Requires a small code change per the retrieval spec §14.

None of these triggers are close at MVP scale. Documenting them so the response is planned rather than improvised when they do fire.

15. Failure modes and recovery

Common failure modes and their responses.

Failure	Detection	Response
Fly region outage	Uptime monitoring fires; Fly status page confirms	Wait for Fly recovery. No action possible from Studium's side. Alert users if outage exceeds 2 hours.
Postgres primary failure	Backend health check fails; Sentry captures connection errors	Fly Postgres auto-failover kicks in if configured; otherwise manual restore from most recent backup.
Anthropic API outage	Sentry captures API errors; per-request degradation copy shows to users	Wait for provider recovery. Retrieval falls back to keyword-only per retrieval §16; agents surface graceful error messages.
Voyage API outage	Retrieval falls back to keyword-only per retrieval §16	Wait for provider recovery. Retrieval quality is degraded but not broken.
Frontend down	Uptime monitoring fires	Rollback if recent deploy caused it; otherwise investigate via Sentry and Fly logs.
Backend down	Uptime monitoring fires (frontend can't reach backend)	Same as above.
Signing key compromise	Human report (nothing detects this automatically)	Emergency rotation per §11.4.

Failure	Detection	Response
Data corruption	Sentry captures inconsistency errors; reviewer notices anomaly	Restore from most recent backup; investigate cause.
Budget cap systematically exceeded	Cost alerts fire	Review usage patterns; either raise caps, tighten per-user limits, or investigate a runaway agent loop.
Unhandled prompt injection	Sentry captures unexpected agent behavior; content review queue fills with anomalies	Analyze specific inputs; add filters or detection at the input boundary; may require prompt hardening.

Every failure has a designed response. No failure results in silent data loss or silent degradation without a queue entry or alert.

16. Testing strategy

Meta-testing plus operational drills.

Tier 1 — Offline.

- CI workflow YAML parses correctly for every subsystem workflow.
- Secret variable names match between production, local, and CI configurations.
- Alert threshold configurations are valid.
- Retention worker algorithm handles edge cases (empty tables, large batches, transaction failures).
- Restore procedure documentation exists and is up to date.

Tier 2 — Online (requires a scratch environment).

- Full deploy pipeline: builds, migrates, deploys, runs smoke test.
- Rollback: deploy, verify, roll back, verify previous version restored.
- Backup restore drill: quarterly, per §9.3.
- Secret rotation: quarterly, per §6.3.
- Retention worker: verify against a seeded database that policies delete correct rows.

Tier 3 — Production (only for tests that require production data or configuration).

- Cost report accuracy: monthly reconciliation of `studium ops cost-report` output against Anthropic and Voyage invoices.
- Uptime SLA measurement: quarterly report of actual uptime from external monitoring.
- Alert firing accuracy: quarterly review of what alerts fired, whether they represented real issues, whether real issues had corresponding alerts.

17. Version history

v1.0 — 25 August 2026. Initial specification. Written against all six prior specs. Locks: single-region Fly.io deployment (Toronto), managed services (Fly Postgres, Cloudflare R2, Langfuse cloud, Sentry SaaS), Fly secrets management with quarterly rotation, four-tier observability (Langfuse LLM, Sentry errors, OpenTelemetry HTTP, Fly infrastructure metrics), calendar-scheduled backups with quarterly restore drills, Ed25519 signing key management with annual rotation, manual deployment promotion for MVP, scaling triggers documented but not automated.

Anticipated v1.1 candidates.

- Staging environment when multi-user usage warrants it.
- Multi-region deployment when geography warrants it.
- Auto-scaling policies once real load patterns are known.
- Paging service integration when uptime SLAs matter.
- Grafana or similar for metric visualization beyond what Fly and Langfuse provide.
- Additional automation for cost anomaly detection.

18. Forward references and open questions

Data layer v1.2 additions from this subsystem. Three items join the batch:

- `retention_actions` table (§12.2)
- `retention_holds` table (§12.4)
- `signing_keys.compromised_at` column (§11.4)

Plus one more implied by §12.2's audit needs — a metadata field on `retention_actions` with structured provenance. Counting: **twenty-two total items in the data layer v1.2 batch when all seven specs' additions are folded in.**

No open questions requiring build-time answers. This is the last spec; there is no subsequent subsystem for open questions to inform. What remains open is measurement — real cost numbers, real latency numbers, real failure patterns — none of which the spec can answer.

Cross-cutting operational questions worth naming.

1. **When does staging become worth adding?** No principled answer. When you find yourself needing to test something you're afraid to test in production, that's the signal.
2. **When does the current alert delivery (email to one person) become insufficient?** When a second operator joins, when uptime becomes financially consequential, or when your response time to alerts becomes the bottleneck for reliability.
3. **When do you outgrow Fly.io?** The realistic answer is "you don't at any scale Studium is likely to reach in the first year." Fly handles up to hundreds of thousands

of concurrent users on modest topology; nothing about Studium's projected scale requires migration to a hyperscaler.

End of specification

This document defines the operational envelope for Studium. Single region, managed services, four-tier observability, scheduled backups with tested restore, annual key rotation, manual deploy promotion, documented scaling triggers. A senior engineer with all six prior specs and this document can deploy the system to Fly.io, connect the observability stack, verify backups, and hand the reviewer credentials and dashboards to run the system day-to-day.

This is the last spec in the seven-subsystem sequence. When the corresponding build lands, and when the pending v1.2/v1.1 revisions batch, the specification set for MVP is complete. What remains after that is measurement, content authoring, and the ongoing operational rhythm the spec now describes.